
Policy-based Reinforcement Learning methods with critic applied to CartPole

Ali Momen (s4993128)* Bram Bouma (s2646072)* Judith Moulijn (s3206025)*

Abstract

This article focuses on the implementation of policy-based methods of Reinforcement Learning for the CartPole problem. The different methods' performance are compared to each other and to a benchmark solution (baseline) and DQN solution, as well. The implemented methods include first **REINFORCE**, then attempting to improve its solution by adding some critic to the learning process using **Actor-Critic** methods **AC** and **A2C**.

1. Introduction

For problems that practically cannot be addressed by tabular methods—such as CartPole with continuous observations and discrete actions or robotic movement control—algorithms using function approximation is the only viable option. Function approximations can be used to train Q -value approximators and/or policy function approximators. If only Q -values are approximated, the action space is discrete or discretized, and the policy estimation is performed through Q -values, the algorithm is called Deep Q -learning (DQN). If the policy is estimated directly by training a network estimator, the algorithm is a **policy-based** algorithm. However, the Q -values can also be trained as a means and **critic** to better adjust and improve the policy training. The **REINFORCE** algorithm purely trains the policy directly, but **AC** and **A2C** algorithms blend in value adjustments. This study is dedicated to the implementation and analysis report of these algorithms, in order to quantify how they compare to each other and how they improve on DQN.

In [section 2](#) the theory necessary for understanding the methods used in this report will be introduced. In [section 3](#) the environment, agent and setup of the experiments is explained. The resulting figures are shown in [section 4](#) and discussed in [section 5](#). We conclude our findings in [section 6](#).

*Equal contribution. Correspondence to: Ali Momen <amomen@gmail.com>, Bram Bouma <bram.f.bouma@gmail.com>, Judith Moulijn <j.moulijn@umail.leidenuniv.nl>.

2. Theory

2.1. REINFORCE

2.1.1. VANILLA LOSS

The term ‘vanilla loss’ is more commonly attributed to the REINFORCE algorithm. When we consider the loss function to be in the simplest form:

$$L = -\mathbb{E}[\log \pi_{\theta}(a | s) \cdot R]$$

where

$$R = G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

is the raw Monte-Carlo return. It is the basic version without extra stabilizers or modifications, such as clipping as in PPO, trust-region constraints as in TRPO or penalties, entropy bonuses, and sophisticated off-policy corrections. It was introduced to optimize a policy **directly**, handle **continuous action spaces** naturally, and avoid needing to differentiate through the environment.

2.1.2. ALGORITHM

The algorithm for the plain REINFORCE approach is shown in [Algorithm 1](#). We have multiple full-episodes (in Monte-Carlo manner) that we use to train our estimation function with. For CartPole, we have to sample episodes that end with falling of the pole (trespassing maximum deviation from a vertical position), and we feed the system with an episode, one at a loop iteration to adjust the training via gradient ascend/descend method. The learning rate α is used to regulate the policy update. To find the best policy the parameters θ must be updated such that the return is maximised.

Algorithm 1 REINFORCE (Monte Carlo Policy Gradient)

Input: policy network $\pi_{\theta}(a | s)$, number of episodes M , learning rate α , discount γ

Initialise: Randomly initialise θ

for $i \in 1, \dots, M$ **do**

 Sample trace $\tau = \{s_0, a_0, r_0, s_1, \dots, s_T\}$

for $t \in 0, \dots, T-1$ **do**

$\hat{Q}(s_t, a_t) = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}$

end for

$\theta \leftarrow \theta + \alpha \cdot \sum_t \hat{Q}(s_t, a_t) \cdot \nabla_{\theta} \log[\pi_{\theta}(a_t | s_t)]$

end for

2.2. AC

The REINFORCE algorithm can be expanded by introducing a neural network for the value function $V_\varphi(s)$ that needs to be estimated using the same Monte-Carlo estimation. The reason for using a value function network $V_\varphi(s)$ instead of a state-action value network $Q_\varphi(s)$ is mostly simplicity and this helps the code be similar to the code for the A2C algorithm (which will be explained later). The policy and value networks are referred to as the actor and the critic respectively. Using this actor-critic (AC) approach utilises the low-bias aspect from REINFORCE due to the random action selection and reduces REINFORCE's high-variance by introducing the critic (Plaet, 2022).

Before, the DQN algorithm was applied to the CartPole environment. This off-policy algorithm used the $\max_{a'} Q(s', a')$ operation to help determine Bellman target that is used in the loss that needed to be minimised. This same loss cannot be used for the AC algorithm however. For this algorithm actions are chosen based on the probability assigned to them by the current policy π_θ . The AC algorithm aims to approximate the value function under the current stochastic policy, not under the optimal policy, which is what the DQN algorithm does. The loss function that is now used to update the actor network weights is

$$L(\theta) = - \sum_t \hat{Q}(s_t, a_t) \log[\pi_\theta(a_t|s_t)]$$

Where $\hat{Q}(s_t, a_t)$ is the Monte-Carlo estimate for $Q^{\pi_\theta}(s_t, a_t)$. The loss function that is used to update the critic network weights is

$$L(\varphi) = \sum_t (\hat{Q}(s_t, a_t) - V_\varphi(s_t))^2$$

Using $V_\varphi(s)$ instead of $Q_\varphi(s, a)$ might seem unintuitive. However, since the two are related via

$$V^{\pi_\theta}(s) = \mathbb{E}_{a \sim \pi_\theta} [Q^{\pi_\theta}(s, a)]$$

using $Q_\varphi(s, a)$ instead would result in approximately the same quantity being minimised. This is only the case since the loss is calculated over many episodes. The pseudocode for this algorithm is shown in Algorithm 2.

Algorithm 2 Actor Critic (AC) algorithm

Input: policy network $\pi_\theta(a|s)$ (actor), value network $V_\varphi(s)$ (critic), number of episodes M , actor learning rate α , critic learning rate β and discount factor γ

Initialise: Randomly initialise θ and φ

for $i \in 1, \dots, M$ **do**

Sample trace $\tau = \{s_0, a_0, r_0, s_1, \dots, s_T\}$

for $t \in 0, \dots, T-1$ **do**

$\hat{Q}(s_t, a_t) = \sum_{k=0}^{T-t-1} \gamma^k \cdot r_{t+k}$

end for

$\theta \leftarrow \theta + \alpha \cdot \sum_t \hat{Q}(s_t, a_t) \cdot \nabla_\theta \log[\pi_\theta(a_t|s_t)]$

$\varphi \leftarrow \varphi - \beta \cdot \sum_t (\hat{Q}(s_t, a_t) - V_\varphi(s_t))^2$

end for

2.3. A2C

The Actor-Critic (AC) approach described above uses a full Monte-Carlo return, taking all the rewards into account until the end of an episode. In contrast to AC, Advantage Actor-Critic (A2C) computes the n -step target. It sums over n rewards and then bootstraps using the critic. This is also the reason why the AC algorithm doesn't include the estimation depth term used in determining $\hat{Q}(s_t, a_t)$. By calculating the Monte-Carlo target, instead of an n -step target, the value of $V_\varphi(s_{t+n})$ equals zero, since s_{t+n} is the terminal state.

Furthermore, while AC updates the actor directly with the target $\hat{Q}$, A2C subtracts the baseline (the value function) to compute the advantage. The advantage represents the value of taking a specific action in a certain state relative to the value of that state, thereby isolating the contribution of the action from the overall quality of the state. This advantage is then used to update the actor, which reduces the variance of the policy gradient updates. The full algorithm for A2C is showed in Algorithm 3

Algorithm 3 Advantage Actor Critic (A2C) algorithm

Input: policy network $\pi_\theta(a|s)$ (actor), value network $V_\varphi(s)$ (critic), number of episodes M , estimation depth $\tilde{n}$, actor learning rate α and critic learning rate β

Initialise: Randomly initialise θ and φ

while not converged **do**

for $i \in 1, \dots, M$ **do**

Sample trace $\tau = \{s_0, a_0, r_0, s_1, \dots, s_T\}$

for $t \in 0, \dots, T-1$ **do**

$\hat{Q}_n(s_t, a_t) = \sum_{k=0}^{n-1} \gamma^k \cdot r_{t+k} + \gamma^n \cdot V_\varphi(s_{t+n})$

$\hat{A}_n(s_t, a_t) = \hat{Q}_n(s_t, a_t) - V_\varphi(s_t)$

end for

$\theta \leftarrow \theta + \alpha \cdot \sum_t \hat{A}_n(s_t, a_t) \cdot \nabla_\theta \log[\pi_\theta(a_t|s_t)]$

$\varphi \leftarrow \varphi - \beta \cdot \sum_t (\hat{A}_n(s_t, a_t))^2$

end for

end while

3. Experiments

In order to test different algorithms, the CartPole environment from the open-source Python library Gymnasium (Towers et al., 2024) is used. Gymnasium provides a standardized Application Programming Interface (API) for developing and testing different RL algorithms and includes several physics simulation environments. The CartPole environment is two-dimensional and consists of a pole placed vertically on a cart with a passive joint. The cart has the ability to move sideways along a frictionless track. The pole acts like a pendulum, with the goal of balancing it for as long as possible by applying forces in the left and right direction onto the cart.

The application of the left and right forces are the two actions that can be taken in the environment and together form the action space. The observation space consists of four observables: The cart position, the cart velocity, the pole angle and the pole angular velocity. An episode is terminated if the angle of the pole is more than 12° or if the position of the cart leaves the $(-2.4, 2.4)$ range. A reward of 1 is given for every timestep that the pole remains upright, with a threshold of 500, due to a maximum of 500 timesteps per episode.

Each algorithm is run for one million timesteps. To get a reliable estimate of the performance, each of the learning curves are averaged over five repetitions. Furthermore, their Gaussian standard deviation is calculated to account for the uncertainty margin.

3.1. REINFORCE

The returns for the REINFORCE algorithm are collected by evaluating its performance every 250 timesteps. Then the curve has been produced. For every hyperparameter, multiple values have been tested to obtain near optimal values that will be further discussed later in the discussion and appendix sections. The optimal values obtained include the following: $\alpha = 0.001$, $\gamma = 0.99$, actor network with hidden layers dimension of $[32, 32]$ (where the length of the list indicates the number of hidden layers and the number indicates the size of each layer), input size 4, and output size 1.

3.2. AC

The returns for the AC algorithm are collected by evaluating its performance every 2500 timesteps. To determine which hyperparameters to use, some values for α , β , actor neural network architecture and critic network architecture were tested. This is shown in the appendix (Appendix B). Based on these tests α is set to 0.001, as is β . The hidden layer of the policy network (the actor) is chosen to be $[64, 64]$. The hidden layer of the value network (the critic) is set to $[128, 128]$ and γ is set to 0.99. The resulting performance is shown in Figure 1.

3.3. A2C

The hyperparameters that are considered to have the greatest effect on the learning process are tuned to result in an optimal learning curve. The first one is the estimation depth $\tilde{n}$, which dictates the number future rewards that are considered in the n -step target, before bootstrapping. Since the policy (actor) and the value function (critic) are each estimated by a separate neural network (NN). They each have their own learning rate and network architecture, which is the number and size of hidden layers. So the policy learning rate, the value function learning rate, the hidden layers of the policy NN, and the hidden layers of the value function NN are also hyperparameters that need to be optimized.

These five hyperparameters were optimized using a systematic approach, where one hyperparameter is changed to different values while the others are kept at a constant value. The performance of the different values for the variable hyperparameters after a million timesteps are then compared to each other to determine which one results in the most optimal and stable learning curve. These comparisons result in the optimal version of the A2C (see Figure 1), which has $\tilde{n} = 10$, $\alpha = 0.0001$, $\beta = 0.01$, a policy architecture of two hidden layers with 64 neurons each, and a value function architecture of two hidden layers with 128 neurons each.

4. Results

Figure 1 shows the learning curves for the three algorithms discussed above compared to DQN, all with optimized hyperparameters.

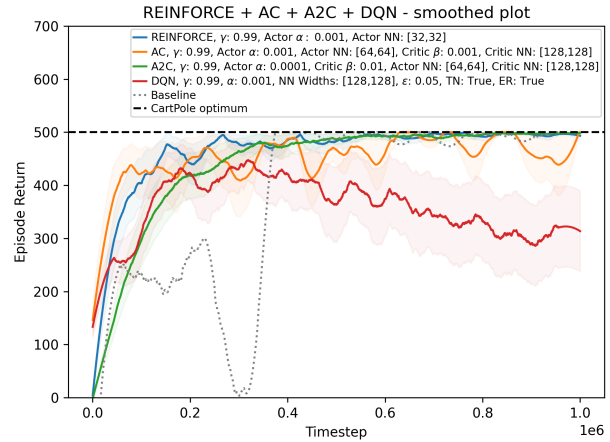


Figure 1. Comparison between the REINFORCE, AC, A2C and DQN algorithms, with optimized hyperparameters. The provided baseline data is also shown. The black dashed line indicates the optimal performance.

5. Discussion

5.1. Improvements from DQN

The DQN algorithm had a network be a part of both the predicted value and the target value used to determine the loss. In order to increase stability and break temporal correlations between states two improvements were made using a target network and replay buffer respectively. These improvements cannot be applied to REINFORCE, AC or A2C however. The training stability that the target network provides for DQN is not necessary in the algorithms covered here since the target and prediction used in the loss do not both depend on the same Q -network. This is the case for DQN.

The replay buffer used actions taken from previous versions of the policy, which worked when the optimal policy needed to be approximated. In the algorithms covered in this work however the current policy π_θ needed to be approximated.

The replay buffer works for off-policy algorithms, like DQN, but not for on-policy algorithms.

5.2. REINFORCE

Compared to baseline and DQN, this algorithm performs better at earlier timesteps. As it reaches higher timesteps, the performance becomes similar to the baseline but excels that of the DQN, and unlike DQN, it does not have an adverse training misbehaviour as the training continues, but it appears that training can be enough with roughly 300,000 time steps. This algorithm performs above expectations, given its simplicity.

5.3. AC

Compared to the baseline data, the AC algorithm performs well at earlier timesteps. As it reaches towards higher timesteps, the performance is similar to that of the baseline data. At these earlier timesteps, the episodes are relatively short which leads to noisy estimates of the gradient. This explains the larger variance seen at these smaller timesteps. The stability of the performance seems to improve as the algorithm is kept running. This is recognised by the reducing frequency of sudden drops in the episode return (which are still seen in the baseline data). For the AC algorithm the evaluation interval is ten times as large as the one used in REINFORCE and A2C. This was an optimisation used to help reduce the runtime in order to reach the one million timesteps. However, this has come at the cost of loss of information.

Compared to DQN, the AC algorithm performs better, since its return keeps increasing after 400,000 timesteps, whilst the DQN's episode return starts to trend downwards. The AC algorithm performs similar to REINFORCE, but starts at a higher episode return in its earlier timesteps. However it does not have the stability that REINFORCE or A2C has, as some dips in the episode return are observed. This can also be an artifact due to the higher evaluation interval and using the same smoothing window. To better compare the algorithms, ideally the same evaluation interval is used.

5.4. A2C

The resulting learning curve of an optimally tuned A2C agent performs consistently better than DQN and the baseline (that is generally considered to solve the CartPole problem in a reasonable amount of timesteps), as can be seen in Figure 1. The A2C learning curve quickly rises well above DQN and the baseline and reaches the optimum return of 500 after about 400,000 timesteps within the uncertainty margin. Then it maintains this optimum with relatively low standard deviation, outperforming the other algorithms. However, before it converges to the optimum, this error margin is significantly larger. This could be improved by averaging over more repetitions than five, but this would come at the cost of an increased computation time.

Follow-up studies could improve A2C further by introducing multiple parallel agents (A3C), constraining the policy change in a single update (PPO) or adding an entropy term to the reward (SAC).

6. Conclusion

With the algorithms covered in this work a comparison can be made between value-based DQN and policy-based REINFORCE, AC and A2C. All three policy-based algorithms out-perform DQN. Whilst REINFORCE and AC both have dips in their return, A2C's performance is almost constantly increasing and stable. Using these algorithms has shown how relatively simple algorithms can outperform the more complicated DQN algorithm, even with the added tricks to improve its episode return.

References

- Plaat, A. *Deep Reinforcement Learning*. Springer Nature Singapore, 2022. ISBN 9789811906381. doi: 10.1007/978-981-19-0638-1. URL <http://dx.doi.org/10.1007/978-981-19-0638-1>.
- Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., Perez-Vicente, R., Pierré, A., Schulhoff, S., Jet Tai, J., Tan, H., and Younis, O. G. Gymnasium: A Standard Interface for Reinforcement Learning Environments. *arXiv e-prints*, art. arXiv:2407.17032, July 2024. doi: 10.48550/arXiv.2407.17032.

Appendix

Similar to before, we tried to realize how and with what pattern does every hyperparameter change behaves, this could help us have a better notion of how we should have approached the optimal value (approach is exclusive for every hyperparameter) instead of blind random guesses. Also, multiple values were tested for each hyperparameter type. Here the plots used to find the optimal hyperparameters are shown.

A. Hyperparameter optimisation REINFORCE

The values for the learning rate that were tested are 0.0005, 0.001 and 0.005. 0.0005 slowed down the training, 0.005 caused the curve to fluctuate and show volatile behaviour which also slowed down the training. For γ the values 0.9, 0.95, 0.99 and 0.999 were tested. Lower values deprived the training from using future rewards which our survival model depends on, whereas overly considering the future rewards. For the neural network hidden layer architecture, the width 32 and the depth 2 as can be seen in the plots perform best.

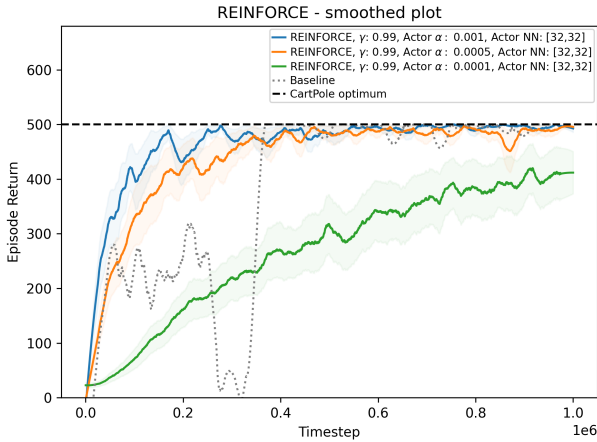


Figure 2. Learning rate (α) in REINFORCE

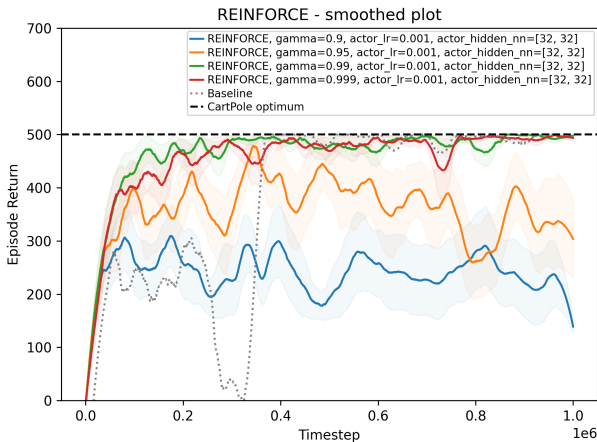


Figure 3. Different γ values for REINFORCE

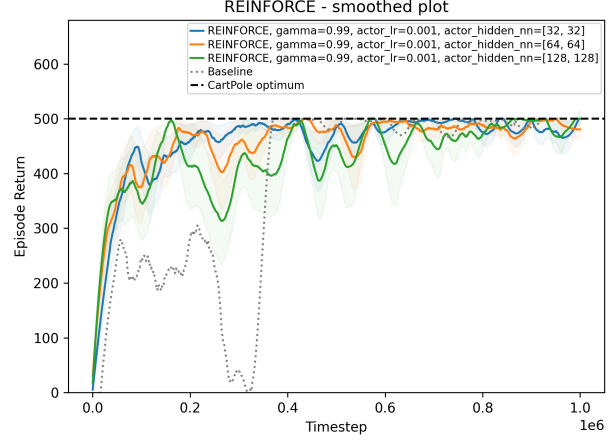


Figure 4. Different Neural Network widths for the Actor network of REINFORCE

B. Hyperparameter optimisation AC

To find the hyperparameters that produce the best performance of the AC algorithm, each hyperparameter of the algorithm is varied. The parameters that are not varied are set to the following baseline values: $\alpha = 0.001$, $\beta = 0.001$, hidden layers of the policy network have the shape [16,16] and the hidden layers of the value network have the shape [64,64].

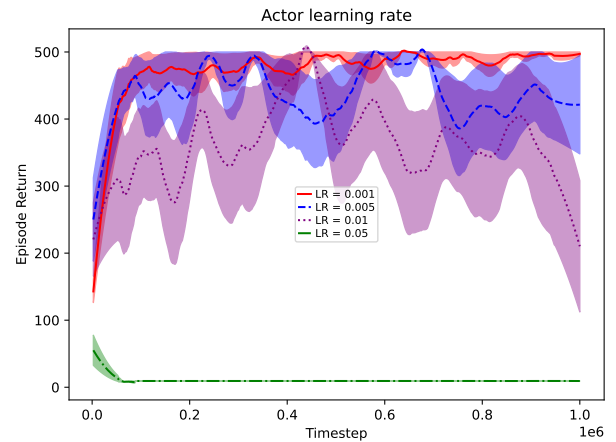


Figure 5. Learning curves of AC algorithm with different values of the actor learning rate α . Each graph is produced using five repetitions. The other hyperparameters are set to baseline values. $\alpha = 0.001$ is chosen as the best performing value.

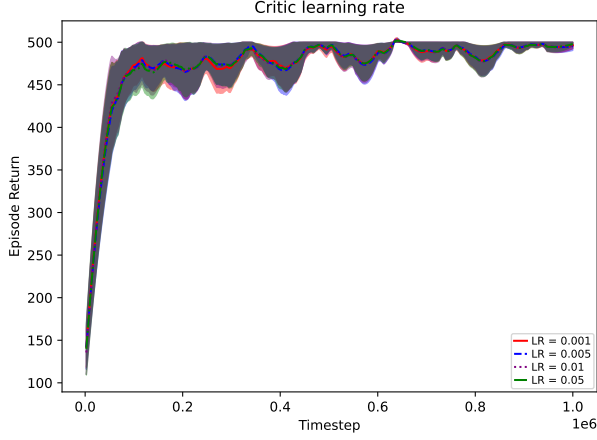


Figure 6. Learning curves of AC algorithm with different values of the critic learning rate β . Each graph is produced using five repetitions. The other hyperparameters are set to baseline values. All learning rates appear to perform similarly, so $\beta = 0.001$ is arbitrarily chosen as the best performing value.

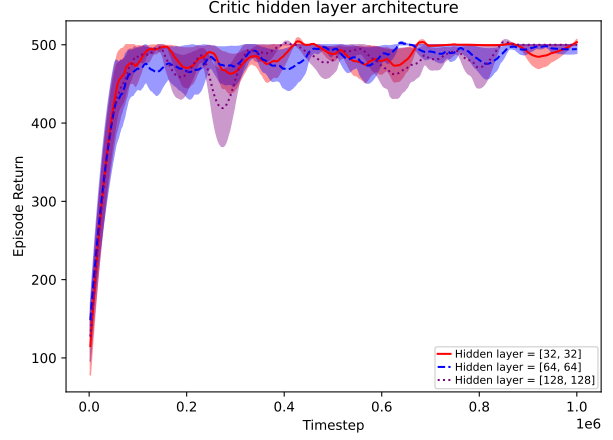


Figure 8. Learning curves of AC algorithm with different shapes of the critic hidden layer. Each graph is produced using five repetitions. The other hyperparameters are set to baseline values. The shape [128,128] is chosen as the best performing one. Likewise this produces the most stable return towards the end of the runs (for the shapes tested).

C. Hyperparameter Optimization A2C

The estimation depth $\tilde{n}$ was tested for the values 1, 5, and 10, the policy learning rate α was tested for the values 0.001, 0.0001 and 0.00001 and the value function learning rate β was tested for the values 0.01, 0.001 and 0.0001. The hidden layer architectures used for optimization were [32], [64, 64] and [128, 128] for both neural networks.

The hyperparameters that remained unvaried during these experiments were an estimation depth $\tilde{n}$ of 5, a learning rate (α and β) of 0.0001 and a hidden layers architecture of [64, 64] for both the policy and the value function neural networks

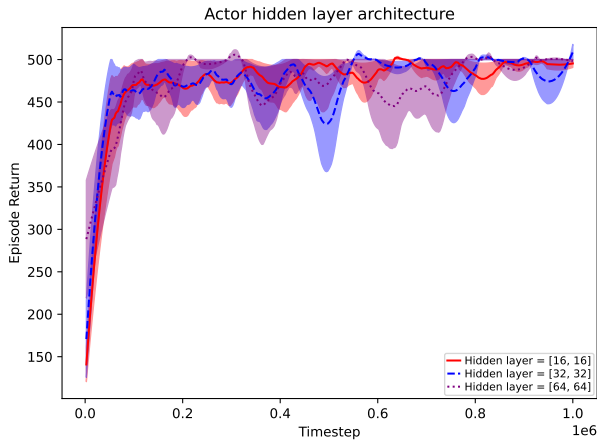


Figure 7. Learning curves of AC algorithm with different shapes of the actor hidden layer. Each graph is produced using five repetitions. The other hyperparameters are set to baseline values. The shape [64,64] is chosen as the best performing one. For this value the return is the most stable towards the end of the runs.

For the estimation depth, the policy learning rate (see Figure 10), and the value function architecture (see Figure 13) the chosen values are clearly the best performing values after one million timesteps. For the value function learning (see Figure 11) rate and the policy architecture (see Figure 12) the distinction between the tested values was less clear. In the case of the value function learning rate, the values of 0.01 and 0.001 both perform roughly equally well, while a learning rate of 0.0001 performs significantly worse. However, 0.01 seemed to have a somewhat lower error margin, suggesting a more stable learning process, even though the difference is small. For the policy architecture, two hidden layers with 128 neurons each results in a very unstable learning curve over many timesteps, with an episode return that spikes and dips without converging. From the two remaining architectures, the one with two hidden layers of 64 neurons each seems to perform consistently better than the one with one hidden layer of 32 neurons.

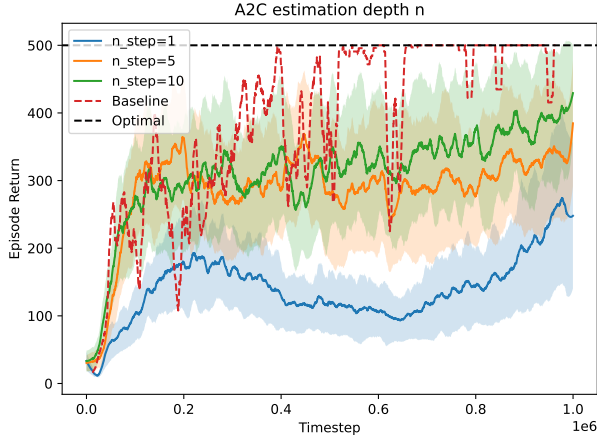


Figure 9. Learning curves of A2C, experimenting with estimation depths, while the remaining hyperparameters are set to their baseline value. The learning curves are averaged over 5 repetitions, with the shaded areas indicating the standard deviation and the red dotted line showing the baseline solution for CartPole.

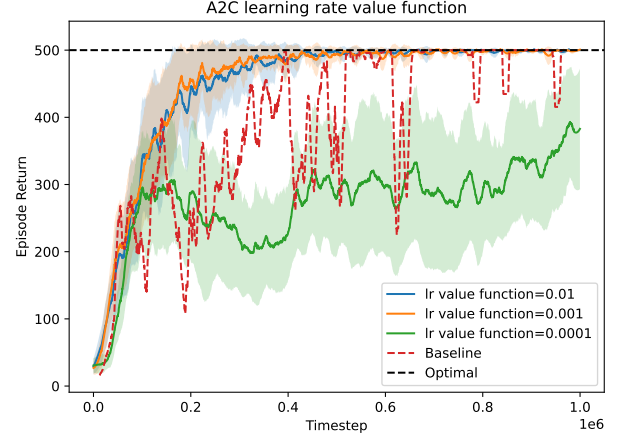


Figure 11. Learning curves of A2C, experimenting with different value function learning rates, while the remaining hyperparameters are set to their baseline value. The learning curves are averaged over 5 repetitions, with the shaded areas indicating the standard deviation and the red dotted line showing the baseline solution for CartPole.

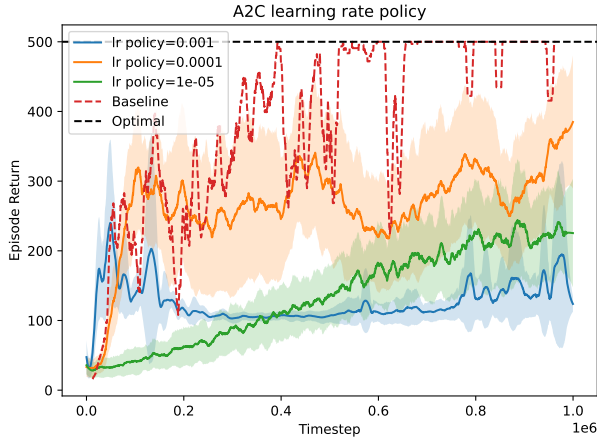


Figure 10. Learning curves of A2C, experimenting with different policy learning rates, while the remaining hyperparameters are set to their baseline value. The learning curves are averaged over 5 repetitions, with the shaded areas indicating the standard deviation and the red dotted line showing the baseline solution for CartPole.

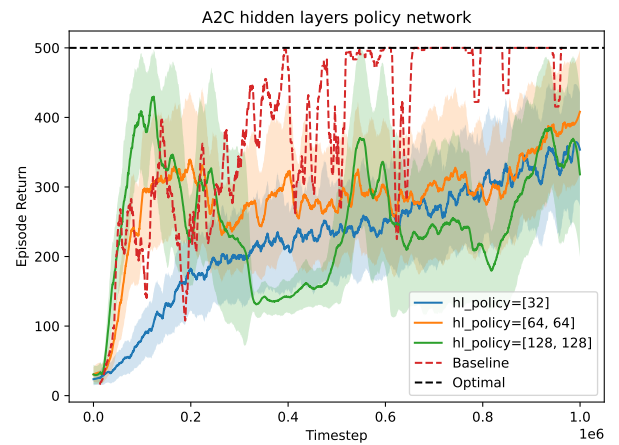


Figure 12. Learning curves of A2C, experimenting with different policy architectures, while the remaining hyperparameters are set to their baseline value. The learning curves are averaged over 5 repetitions, with the shaded areas indicating the standard deviation and the red dotted line showing the baseline solution for CartPole.

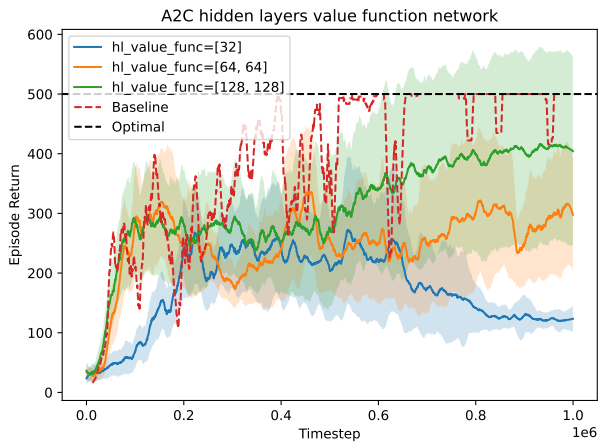


Figure 13. Learning curves of A2C, experimenting with different value function architectures, while the remaining hyperparameters are set to their baseline value. The learning curves are averaged over 5 repetitions, with the shaded areas indicating the standard deviation and the red dotted line showing the baseline solution for CartPole.